

DD: BIL-02-READ-01 — Invoice Read API

Developer implementation guide: DD_BIL-02-READ-01-Invoice_Read_API-DEV_IMPL_GUIDE-v1.5.md.

1. Purpose

This rewritten DD is the developer-facing version of the module contract
DD_BIL-02-READ-01-Invoice_Read_API-v1.5.md.

Verdict: ■ New | **Wave:** 1 | **Group II:** Billing & Revenue | **Version:** v1.5 Satellite of **BIL-02 Invoicing (Core)**. Owns the read-side surface for invoices — the unified retrieval API (list, detail, status, PDF stream) consumed by the customer self-service portal, mobile app, admin console, and B2B integrations. Performance-critical satellite: high-traffic, customer-facing, requires Redis caching. PDF rendering and channel-based delivery (email, SMS, WhatsApp, Telegram, push) are owned by **DD_NOT-01 Notification Service**, not by this DD. **Sibling DDs:** BIL-02-GEN-01 Invoice Generation, BIL-02-TAX-01 Tax Invoice & Gateway, BIL-02-ADJ-01 Invoice Adjustments, BIL-02-STA-01 Status Management & Recovery. **Version history:** see CHANGELOG_BIL-02-READ-01-Invoice_Read_API.md in the same directory.

The goal of this rewrite is to make the DD easier for a mid-level developer to implement without losing the detailed source contract.

2. Implementation Scope

This DD should be implemented as part of the owning SOPHIX capability, not as an isolated service unless the deployment architecture explicitly separates that capability.

Implement this DD with these rules:

- Use the owning module APIs/repositories for authoritative writes.
- Use approved internal APIs/client wrappers when reading another module's data.
- Do not query another module's database tables directly from business flow code.
- Treat worker names as logical Camunda external-task topics, not separate deployments.
- One worker application may handle multiple topics, but metrics, retries, and logs must remain visible per topic.
- Emit success events only after the authoritative state commit succeeds.
- Use outbox publishing for Kafka events.

3. Runtime Metadata

Item	Value
Source file	/Users/macbook/Downloads/Sophix_V3_BSS_DDs_MD_2026-05-27/05_billing/DD_BIL-02-READ-01-Invoice_Read_API-v1.5.md
Version	v1.5
DD type	module contract
BPMN/process keys	Not explicitly defined
Drools packages	Not explicitly defined

4. Runtime Contracts To Implement

4.1 APIs

- GET /api/admin/invoices/download-stats?operator={code}
- GET /api/admin/invoices/pdf-render-failures?operator={code}
- GET /api/invoices
- GET /api/invoices/credit-notes
- GET /api/invoices/cycle
- GET /api/invoices/debit-notes

- GET /api/invoices/inv_01HZQX8N5V8KJEC9M3T2A4DPYW
- GET /api/invoices/inv_01HZQX8N5V8KJEC9M3T2A4DPYW/pdf
- GET /api/invoices/inv_01HZQX8N5V8KJEC9M3T2A4DPYW/status
- GET /api/invoices/pf_01HZX9M2K3LA8N7QR4S5T6U7VW
- GET /api/invoices/pro-forma
- GET /api/invoices/tax
- GET /api/invoices/{id}
- GET /api/invoices/{id}/pdf
- GET /api/invoices/{id}/status
- GET /api/invoices/{id}?lightweight=true
- GET /api/invoices?account_id=X&status=PENDING&sort=issue_date_desc&limit=20
- GET /api/invoices?operator_code=WIK&customer_id=cust_01HX...&issue_date_from=2026-01-01&issue_date_to=2026-05-31&limit=50
- GET /api/invoices?status=PENDING&sort=issue_date_desc&limit=20
- POST /api/admin/cache/invalidate
- POST /api/admin/invoices/inv_01HZQX8N5V8KJEC9M3T2A4DPYW/rerender-pdf
- POST /api/admin/invoices/{id}/rerender-pdf

4.2 Tables

- No CREATE TABLE block is explicitly declared in this DD.

For each table in this DD, implement the schema with:

- a short table comment explaining what the table is for;
- column comments or migration notes explaining each field;
- constraints and indexes from the DD;
- seed data where the DD provides operator-specific sample rows.

4.3 Worker Topics

- billing-invoicing
- billing-invoicing-read

Worker-topic guidance:

- A BPMN service task uses the topic.
- A worker application subscribes to the topic.
- The topic handler validates input variables, calls the owning module/API, writes outputs back to Camunda, and records metrics.
- Do not treat the topic string as a shell command or Kubernetes deployment name.

4.4 Events

- InvoiceCancelled
- InvoiceIssued
- InvoiceStatusChanged
- PaymentApplied

Event guidance:

- Write events through the transactional outbox.
- Include operation id, aggregate id, operator code, actor, and correlation data.

- Consumers should be able to rebuild downstream state without querying workflow internals.

5. Developer Implementation Checklist

1. Add or verify database migrations and seed rows.
2. Add or verify config rows for the operator/module.
3. Implement request/command DTOs and validation.
4. Implement idempotency and in-flight operation checks where the DD requires them.
5. Implement Drools fact mapping and package deployment where rules are used.
6. Implement BPMN process, service tasks, gateways, timers, and message catches where the DD is a flow.
7. Implement worker-topic handlers using owning module APIs.
8. Implement compensation paths and manual recovery paths.
9. Emit success/rejected events through the outbox.
10. Add smoke tests for happy path, validation failure, dependency failure, and retry/idempotency.

6. Infrastructure And AWS Notes

Deploy this DD with the existing SOPHIX platform components unless the owning module requires isolation:

Concern	Recommendation
API/runtime	Run inside the owning module deployment or existing workflow API pods.
Workers	Consolidate low-volume topics into shared worker apps; keep per-topic metrics.
Database	Use the owning module PostgreSQL schema and migration pipeline.
Kafka	Use the foundation Kafka/MSK setup and transactional outbox.
Camunda	Deploy BPMN files through the workflow deployment pipeline.
Drools	Deploy KJAR/rule artifacts through the approved rules pipeline.
Secrets	Use the platform secret manager; on AWS prefer Secrets Manager/Parameter Store with IRSA.
Observability	Alert on stuck operations, failed workers, outbox backlog, and external dependency failures.

7. Original DD Detail Preserved

The following section preserves the detailed source contract for implementation and review.

Verdict: ■ New | **Wave:** 1 | **Group II:** Billing & Revenue | **Version:** v1.5

Satellite of **BIL-02 Invoicing (Core)**. Owns the read-side surface for invoices — the unified retrieval API (list, detail, status, PDF stream) consumed by the customer self-service portal, mobile app, admin console, and B2B integrations. Performance-critical satellite: high-traffic, customer-facing, requires Redis caching. PDF rendering and channel-based delivery (email, SMS, WhatsApp, Telegram, push) are owned by **DD_NOT-01 Notification Service**, not by this DD.

Sibling DDs: BIL-02-GEN-01 Invoice Generation, BIL-02-TAX-01 Tax Invoice & Gateway, BIL-02-ADJ-01 Invoice Adjustments, BIL-02-STA-01 Status Management & Recovery.

Version history: see `CHANGELOG_BIL-02-READ-01-Invoice_Read_API.md` in the same directory.

Glossary

Term	Meaning
Read API	The HTTP surface this DD owns: list, detail, status, and PDF stream endpoints for invoices of any document type. Pull-based — initiated by the consumer (customer, admin, integration).

Term	Meaning
Document type	The invoice type code as defined in BIL-02 Core's <code>invoice_type_definition</code> : <code>CYCLE_POSTPAID</code> , <code>ONE_OFF</code> , <code>PRO_FORMA</code> , <code>CREDIT_NOTE</code> , <code>DEBIT_NOTE</code> , <code>TAX_INVOICE</code> . READ-01 serves all of them through unified endpoints.
PDF artifact	The rendered PDF representation of an invoice. Produced by DD_NOT-01 Notification Service when the invoice is issued, using per-operator templates; stored in object storage; the URL is written back to <code>invoice.metadata.pdf_url</code> . READ-01 streams the stored file when asked. READ-01 does NOT render and does NOT own templates.
Customer-scoped access	A consumer authenticated as a customer can only see invoices where <code>invoice.customer_id</code> matches their JWT identity. Enforced on every read.
Operator-scoped access	A consumer authenticated as an admin can only see invoices for the operator(s) they have a role in. Enforced on every admin read.
Integration-scoped access	A B2B integration (e.g., accounting platform) authenticated with an integration JWT; sees invoices for the customer IDs allowed in the JWT's scope.
Cache tier	The Redis cache layer READ-01 uses for high-traffic customer-facing GETs. Configured per the patterns in <code>FOUNDATION_CACHE.md</code> . Invalidated on lifecycle events affecting the cached invoice.

A — Target

1. What this process is about

READ-01 owns the read-side surface for invoices: list, detail, status, and PDF stream endpoints consumed by the customer portal, mobile app, admin console, and B2B integrations.

Reads are served from BIL-02 Core's tables. **Invoice-type documents** (cycle, one-off, credit/debit notes, tax invoices) live in `invoice`. **Pro forma documents** live in `pro_forma` (separate table by design — pro forma are informational projections, not legal documents). READ-01 reads both and exposes them through unified endpoints, with the response payload reflecting which table the row came from.

The boundary with DD_NOT-01 Notification Service

Concern	Owned by
Invoice list / detail / status retrieval (JSON)	READ-01
PDF stream retrieval (binary stream from object storage)	READ-01
PDF rendering using per-operator templates	DD_NOT-01 Notification Service
Storage of rendered PDF artifacts	DD_NOT-01 Notification Service (writes); READ-01 reads
Per-channel templates (email, SMS, WhatsApp, Telegram, push)	DD_NOT-01 Notification Service
Push delivery dispatch and retry on failure	DD_NOT-01 Notification Service
Caching of read responses	READ-01
Access control on reads	READ-01
Download audit log	READ-01

What "downloading a PDF" actually means in this DD

The PDF a consumer retrieves via READ-01 is **not generated by READ-01**. It was rendered by DD_NOT-01 Notification Service at issuance (consuming events like `InvoiceIssued` / `CreditNoteIssued` / `TaxInvoiceSigned`), stored in object storage, with its URL written to `invoice.metadata.pdf_url`. NOT-01 also sent that same PDF (or a link to it) to the customer through configured channels (email/SMS/etc.) at issuance.

READ-01's "download" endpoint simply re-fetches that stored artifact. The customer or admin who already received the PDF at issuance can re-retrieve it any time.

The typical flow — basics before the deep dive

Reading this DD in order, this is what actually happens 99% of the time:

1. **At issuance:** GEN-01 (or TAX-01) writes the invoice row and emits `InvoiceIssued`. NOT-01 picks up the event, renders the PDF using the operator's template, stores it in object storage, writes the URL to `invoice.metadata.pdf_url`, and dispatches the customer notification (email/SMS/WhatsApp/etc. with the PDF or a link to it). All of this happens automatically, within seconds of the invoice being generated. The customer doesn't do

anything — they just receive the notification.

2. **Later, when the customer or admin wants to re-retrieve the PDF:** they hit GET /api/invoices/{id}/pdf. READ-01 reads metadata.pdf_url, fetches the file from object storage, streams it back. **200 OK, done.** No request-to-render, no wait.

The customer never explicitly requests rendering. Rendering is automatic. The download endpoint just serves what's already there.

Exception cases (the remaining 1%)

Three cases where the simple flow doesn't apply:

Case	When it happens	What the consumer sees	Recovery
Race window	Consumer hits the PDF endpoint within the few seconds between invoice issuance and NOT-01 finishing the render	202 Accepted with pdf_status: PENDING and retry_after_seconds: 5	Consumer waits and retries — NOT-01 is already rendering; the next attempt usually succeeds
Operator chose not to render this type	For pro forma in operators that don't produce pro forma PDFs (rare, operator-configurable)	200 OK with pdf_status: UNAVAILABLE and a clear reason	No recovery needed — by design. The document data is available via the detail endpoint; just no PDF for it.
Render failed irrecoverably	NOT-01 couldn't render (e.g., template bug, malformed data) and gave up after retries	503 Service Unavailable with pdf_status: RENDER_FAILED and reason	Admin triggers re-render via POST /api/admin/invoices/{id}/render-pdf — this emits PdfRenderRequested to NOT-01, which retries with fresh inputs. Customer is not expected to do anything; ops handles it.

So: **only admins can trigger rendering**, and only as recovery from failure. There is no consumer-initiated rendering. The customer experience is "click download, get PDF" — anything else is exception handling.

PDF availability by document type

Document type	When PDF is rendered	READ-01 behavior
Cycle invoice, one-off invoice, credit note, debit note	At issuance by NOT-01	Streams the stored PDF
Tax invoice	After the gateway signs it (SIGNED state) — at that point NOT-01 renders with the signed reference and QR code	Streams the stored PDF once SIGNED
Pro forma	Per operator config — some operators render at projection time, others don't render at all (pro forma is purely informational)	Streams stored PDF if present; otherwise returns pdf_status = UNAVAILABLE with explanation
Anything where rendering hasn't completed yet (race window between event and NOT-01 finishing)	—	Returns pdf_status = PENDING with retry hint
Anything where rendering failed irrecoverably	—	Returns pdf_status = UNAVAILABLE; ops investigates via NOT-01

What READ-01 explicitly does not own

- **Generation** (cycle, one-off, pro forma, credit/debit notes) — BIL-02-GEN-01 Invoice Generation
- **Tax invoice generation and signing** — BIL-02-TAX-01 Tax Invoice & Gateway
- **Adjustment proposal/approval workflow** — BIL-02-ADJ-01 Invoice Adjustments
- **Lifecycle state transitions** — BIL-02-STA-01 Status Management & Recovery
- **PDF rendering and per-operator PDF templates** — DD_NOT-01 Notification Service
- **Email / SMS / WhatsApp / Telegram / push notification dispatch** — DD_NOT-01 Notification Service
- **Templates for non-PDF channels** — DD_NOT-01 Notification Service
- **Payment recording or balance updates** — BIL-01 Charging Engine
- **Wallet operations** — BIL-05 Wallet and Topup

2. Business rules and read flow

Rules grouped by phase: L (listing), D (detail), P (PDF), A (access control), C (caching), O (admin operations).

Rule group L — Listing

ID	Rule	Triggered on
R-READ-01-L-1	The list endpoint GET /api/invoices returns a paginated list of documents from both invoice and pro_forma tables (the response payload indicates which source via document_kind field). Filtered by query parameters: account_id (preferred operational scope), customer_id (identity search/grouping scope), subscription_id, invoice_type_code, status, period_start_from, period_start_to, issue_date_from, issue_date_to, payment_status, outstanding_amount_min, outstanding_amount_max. Customer-scoped sessions derive allowed accounts from JWT/ILM context. Operator-scoped admin requests can filter by operator_code, account_id, and customer_id within allowed operators.	Standard listing
R-READ-01-L-2	Default ordering: issue_date DESC. Override via sort query param: issue_date_asc, total_amount_desc, total_amount_asc. Stable secondary sort by id for deterministic pagination.	Sorting
R-READ-01-L-3	Pagination: cursor-based (?after_cursor=...&limit=20). Default limit 20, max 100. Page response includes next_cursor (null when no more) and total result count up to a cap (max 10,000; beyond that returns null to avoid expensive COUNT queries).	Pagination
R-READ-01-L-4	List response is a slim projection: id, invoice_type_code, legal_invoice_number, issue_date, due_date, period_start, period_end, total_amount, currency, status, payment_status, outstanding_amount, pdf_available (bool — whether metadata.pdf_url is set), subscription_id. Line items NOT included — those are fetched via the detail endpoint.	Slim projection
R-READ-01-L-5	Customer-scoped list responses automatically exclude invoices with status = CANCELLED UNLESS ?include_cancelled=true. Admin-scoped responses include all statuses by default.	Default filter on customer view
R-READ-01-L-6	Type-specific list endpoints exist as convenience: GET /api/invoices/cycle, GET /api/invoices/tax, GET /api/invoices/credit-notes, etc. These are equivalent to the generic endpoint with the invoice_type_code filter pre-applied. Customer-facing UIs typically use the generic endpoint with filters; type-specific endpoints exist for integrations that want clean per-type streams.	Type-specific convenience

Rule group D — Detail

ID	Rule	Triggered on
R-READ-01-D-1	The detail endpoint GET /api/invoices/{id} returns the full document. For invoice-type documents: header + line items + customer_snapshot + tax_summary + grouping context + original_invoice_id (if applicable) + pdf_url + signed_invoice_number (for SIGNED tax invoices). For pro forma documents: equivalent projection sourced from pro_forma and pro_forma_line_item tables. Customer-snapshot PII is filtered per the consumer's role.	Standard detail
R-READ-01-D-2	Detail response includes a list of related documents : any credit/debit notes against this invoice (for cycle/one-off), the tax invoice(s) generated from payments against this invoice (for cycle/one-off), the parent invoice (for credit/debit notes and tax invoices). Each related document includes id, invoice_type_code, legal_invoice_number, total_amount.	Related document linking

ID	Rule	Triggered on
R-READ-01-D-3	Customer-scoped detail responses redact admin-only fields: <code>triggering_event_ref</code> , internal metadata keys flagged as internal, full <code>customer_snapshot.audit_metadata</code> . Admin-scoped responses include everything per the admin's role.	Field filtering by role
R-READ-01-D-4	Detail responses include a <code>status_history</code> summary: the lifecycle transitions this invoice has been through (<code>status</code> , <code>transitioned_at</code> , <code>transitioned_by</code>) — sourced from BIL-02 Core's <code>invoice_history</code> . Customer-scoped views show only customer-relevant transitions (e.g., omitting internal flag changes).	Status history
R-READ-01-D-5	If the invoice has <code>pdf_url</code> = NULL (DD_NOT-01 Notification Service hasn't rendered yet, or render failed), the detail response includes <code>pdf_status</code> = PENDING and an estimated availability hint. If rendering has failed irrecoverably, <code>pdf_status</code> = UNAVAILABLE and the response includes a human-readable hint.	PDF status visibility
R-READ-01-D-6	A <code>?lightweight=true</code> variant returns only the header (no line items, no related docs) for fast UI rendering (e.g., the "my invoices" list page that shows summaries). Used by clients to reduce payload size when full detail isn't needed.	Lightweight variant

Rule group P — PDF

ID	Rule	Triggered on
R-READ-01-P-1	The PDF endpoint GET <code>/api/invoices/{id}/pdf</code> streams the stored PDF artifact. READ-01 reads <code>invoice.metadata.pdf_url</code> (or <code>pro_forma.metadata.pdf_url</code>), fetches the file from object storage, and returns it with <code>Content-Type: application/pdf</code> and <code>Content-Disposition: attachment; filename="{legal_invoice_number_or_pro_forma_ref}.pdf"</code> . READ-01 does NOT render — it serves what DD_NOT-01 Notification Service has produced and stored.	Standard PDF download
R-READ-01-P-2	PDF availability varies by document type per the table in section 1. For invoice-type documents in normal state, the PDF is available after NOT-01 finishes rendering (typically seconds after issuance). For tax invoices, available after SIGNED. For pro forma, depends on operator config. If <code>pdf_url</code> is NULL because rendering hasn't completed yet, the endpoint returns 202 Accepted with <code>pdf_status</code> = PENDING and a <code>retry_after_seconds</code> hint.	PDF not yet ready
R-READ-01-P-3	If <code>pdf_url</code> is NULL because no PDF is produced for this document type (e.g., pro forma in operators that don't render pro forma), the endpoint returns 200 OK with <code>pdf_status</code> = UNAVAILABLE and a clear reason. If rendering has failed irrecoverably (NOT-01 escalated to manual review), the endpoint returns 503 Service Unavailable with the recovery path (admin operation on NOT-01, exposed via R-READ-01-O-2).	PDF unavailable or render failed
R-READ-01-P-4	PDF responses include ETag and <code>Cache-Control: private, max-age=31536000, immutable</code> . Once rendered, the PDF artifact is immutable (any re-issue produces a new invoice row with a new PDF URL). Browsers and CDNs can cache aggressively.	Caching headers
R-READ-01-P-5	READ-01 supports range requests (<code>Range: bytes=...</code>) for progressive PDF download — useful for mobile clients on slow connections.	Range requests
R-READ-01-P-6	READ-01 logs every PDF download with: <code>invoice_id</code> , requesting user, scope, timestamp, source IP, user agent. Stored in <code>invoice_download_log</code> (10-year retention).	Download audit log

ID	Rule	Triggered on
R-READ-01-P-7	Admin downloads of PDFs go through the same endpoint as customer downloads — same path, different JWT scope. ACL applies per the access control rules.	Admin uses the same endpoint

Rule group A — Access control

ID	Rule	Triggered on
R-READ-01-A-1	Every endpoint requires authentication. The JWT carries either a <code>customer_id</code> claim (customer-scoped), or a <code>user_id</code> + <code>roles</code> + <code>operator_codes</code> claims (admin/staff), or an <code>integration_id</code> + <code>customer_ids</code> claim list (B2B integration). Anonymous access is not supported.	Authentication
R-READ-01-A-2	Customer-scoped access: the JWT's <code>customer_id</code> MUST equal the invoice's <code>customer_id</code> . List endpoints automatically filter; detail and PDF endpoints return 404 Not Found (NOT 403, to avoid leaking existence) if the customer doesn't own the invoice.	Customer isolation
R-READ-01-A-3	Admin-scoped access: the JWT's <code>operator_codes</code> must include the invoice's <code>operator_code</code> . Roles within an operator govern what fields are returned (e.g., <code>BILLING_ADMIN</code> sees all; <code>CUSTOMER_SUPPORT_AGENT</code> sees standard fields without financial internals).	Operator + role scoping
R-READ-01-A-4	B2B / integration access: a separate JWT with <code>integration_id</code> and <code>customer_ids</code> (the list of customer IDs the integration is allowed to query). Allows accounting integrations to fetch invoices for multiple customers under one API key.	B2B access
R-READ-01-A-5	All access decisions are logged in the standard auth audit trail per <code>FOUNDATION_AUTH.md</code> . PDF downloads additionally log to <code>invoice_download_log</code> per R-READ-01-P-6.	Audit
R-READ-01-A-6	Rate limiting per JWT identity: customer JWTs limited to 100 requests/minute; admin JWTs limited per role; integration JWTs limited per integration tier. Excess returns 429 with <code>Retry-After</code> .	Rate limiting

Rule group C — Caching

ID	Rule	Triggered on
R-READ-01-C-1	READ-01 uses Redis for all customer-facing GETs per <code>FOUNDATION_CACHE.md</code> . Cache keys: <code>invoice:detail:{invoice_id}</code> (full detail), <code>invoice:list:{customer_id}:{filter_hash}:{cursor}</code> (list pages), <code>invoice:status:{invoice_id}</code> (lightweight status).	Cache layer
R-READ-01-C-2	Cache TTLs: detail = 5 minutes, list = 1 minute, status = 30 seconds. Short enough to reflect changes in near-real-time; long enough to absorb traffic bursts. Configurable per operator.	TTLs
R-READ-01-C-3	Cache invalidation is event-driven via Kafka consumption. READ-01 consumes <code>InvoiceCancelled</code> , <code>InvoiceStatusChanged</code> , <code>PaymentApplied</code> , <code>CreditNoteIssued</code> , <code>DebitNoteIssued</code> , <code>TaxInvoiceSigned</code> , <code>PdfReady</code> (emitted by NOT-01 when render completes), and on receipt invalidates the affected invoice's detail, list pages (by <code>customer_id</code>), and status keys.	Event-driven invalidation
R-READ-01-C-4	PDF files are NOT cached in Redis — they're served from object storage with CDN-friendly cache headers (R-READ-01-P-4). The Redis cache is for the structured invoice data only.	PDF cache exclusion

ID	Rule	Triggered on
R-READ-01-C-5	Admin-facing GETs are NOT cached by default (lower volume, fresh data more important). Admin can opt-in to caching by adding <code>?cache=true</code> for read-only operational dashboards.	Admin no-cache default
R-READ-01-C-6	Cache miss path: READ-01 reads from the database, populates Redis with the result, returns to the caller. Cache stampede protection via per-key locks (only one concurrent miss fetches; others wait on the lock).	Miss handling

Rule group O — Admin operations

ID	Rule	Triggered on
R-READ-01-O-1	Admin console provides operational endpoints: GET <code>/api/admin/invoices/download-stats?operator={code}</code> (per-day download counts and bandwidth), GET <code>/api/admin/invoices/pdf-render-failures?operator={code}</code> (lists invoices where DD_NOT-01 Notification Service failed to render).	Operational visibility
R-READ-01-O-2	Admin can trigger a PDF re-render for an invoice where rendering failed: POST <code>/api/admin/invoices/{id}/render-pdf</code> . This emits a <code>PdfRerenderRequested</code> event consumed by DD_NOT-01 Notification Service, which retries rendering. READ-01 does not re-render itself; it just triggers NOT-01.	Re-render trigger
R-READ-01-O-3	Admin can clear cache for a specific invoice or for a customer's entire list cache: POST <code>/api/admin/cache/invalidate</code> . Used by ops staff during incident response.	Cache management
R-READ-01-O-4	Per-operator reporting: top-downloaded invoices per period, distribution of downloads by hour-of-day (capacity planning), customer self-service vs admin download split.	Reporting

3. Module owner and impacted modules

Role	Module	What it does
Owner	New billing-invoicing-read package within the billing-invoicing module on cluster C	Owns all read endpoints, the Redis cache layer integration, the access control enforcement, the PDF streaming proxy from object storage, and the download audit log.
Object storage	Existing S3-compatible object store (per FOUNDATION_DB.md or a separate infra component)	Holds rendered PDF artifacts written by DD_NOT-01 Notification Service. READ-01 reads via signed URLs or direct streaming.
Cache	Redis (per FOUNDATION_CACHE.md)	Caches detail, list, and status responses for customer-facing GETs.
Upstream events consumed	BIL-02 Core (InvoiceCancelled, InvoiceStatusChanged, etc.), BIL-01 Charging Engine (PaymentApplied), GEN-01 (CreditNoteIssued, DebitNoteIssued), TAX-01 (TaxInvoiceSigned), DD_NOT-01 Notification Service (PdfReady, PdfRenderFailed)	Cache invalidation triggers
Downstream events emitted	PdfRerenderRequested (admin-triggered re-render)	Consumed by DD_NOT-01 Notification Service
Frontend	Customer self-service portal, mobile app, admin console — all consume READ-01 endpoints	READ-01 is the backend; the frontends are owned by their respective teams
Engines	No Drools, no Camunda. Pure read-side service.	—

4. Foundation references

DB → FOUNDATION_DB.md · Kafka → FOUNDATION_KAFKA.md · Cache → FOUNDATION_CACHE.md · Auth → FOUNDATION_AUTH.md (relies on existing customer JWT and admin role model). No new auth roles introduced by READ-01.

Technical design

Module placement

Co-located in billing-invoicing module on cluster C. New sub-package read. Adds 1 new dedicated table (invoice_download_log) for audit. All read endpoints share this single module so the cache, ACL, and projection logic are centralized.

Code structure

```
modules/billing-invoicing/  
  src/main/java/com/sophix/billing/invoicing/read/  
    rest/  
      InvoiceReadResource.java      ← unified list + detail endpoints  
      InvoicePdfResource.java       ← PDF download endpoint  
      InvoiceAdminReadResource.java ← admin operational endpoints  
    service/  
      InvoiceReadService.java       ← query orchestration with cache  
      InvoicePdfService.java        ← PDF streaming from object store  
      InvoiceCacheService.java      ← Redis read/write/invalidate  
      InvoiceAccessControlService.java ← customer/admin/integration scoping checks  
    cache/  
      InvoiceCacheKeys.java         ← key conventions  
      InvoiceCacheInvalidator.java ← Kafka-driven invalidation  
    kafka/  
      InvoiceLifecycleEventConsumer.java ← consumes cache-invalidating events  
    audit/  
      InvoiceDownloadLogger.java     ← records PDF downloads  
    domain/  
      InvoiceProjection.java         ← slim list-row projection  
      InvoiceDetailProjection.java   ← full detail projection  
      InvoiceListFilter.java  
      AccessScope.java             ← CUSTOMER / ADMIN / INTEGRATION  
    repository/  
      InvoiceReadRepository.java     ← read-optimized queries against `invoice`  
      ProFormaReadRepository.java    ← read-optimized queries against `pro_forma`  
      InvoiceDownloadLogRepository.java  
  src/main/resources/  
    config/liquibase/changelog/billing-invoicing/  
      070_invoice_download_log.xml
```

Read flow — list endpoint

Client → GET /api/invoices?account_id=X&status=PENDING&sort=issue_date_desc&limit=20

Step 1 – Auth & validation

- Validate JWT
- Extract scope: customer_id (CUSTOMER), operator_codes (ADMIN), or integration_id + customer_ids (INTEGRATION)
- Validate filters: ensure customer_id matches JWT for CUSTOMER scope, operator_code is allowed for ADMIN, etc.

Step 2 – Cache lookup

- Build cache key: invoice:list:{customer_id}:{sha256(filter+sort+cursor)}:{limit}
- GET from Redis
- If hit → return cached payload (typical hot-path latency < 5ms)
- If miss → acquire per-key lock (cache stampede protection); continue to Step 3

Step 3 – Database query

- InvoiceReadRepository.findByFilter(filter, cursor, limit)
- Read-optimized query against `invoice` table with proper indexes
- Returns slim projection per R-READ-01-L-4
- For CUSTOMER scope, query already filters by customer_id; for ADMIN, by operator_code(s)

Step 4 – Cache write

- Serialize result; write to Redis with TTL = 1 min (per R-READ-01-C-2)
- Release lock

Step 5 – Return response

- JSON body: { items: [...], next_cursor: "...", total_count: N }
- Total typical p99 latency: 50ms on miss, 5ms on hit

Read flow — detail endpoint

Client → GET /api/invoices/{id}

Step 1 – Auth & validation

- Same as list

Step 2 – Cache lookup

- Key: invoice:detail:{invoice_id}
- GET from Redis
- If hit → ACL re-check (cached payload includes operator_code, customer_id);
if not matching scope → 404, else return

Step 3 – Database query (on miss)

- Read invoice row + line items + customer_snapshot + tax_summary
- Find related documents (credit/debit notes by original_invoice_id; tax invoices by parent reference)
- Fetch status_history from invoice_history
- Apply field redaction per consumer role (R-READ-01-D-3)

Step 4 – Cache write

- TTL = 5 min
- Release lock

Step 5 – Return response

- JSON body with full invoice detail, related documents, status history, pdf_status

Read flow — PDF download

Client → GET /api/invoices/{id}/pdf

Step 1 – Auth & validation

- Same as detail

Step 2 – Detail read (cached)

- Reuses the detail cache path to get invoice header
- Check invoice.metadata.pdf_url
 - NULL → 202 Accepted with retry hint (R-READ-01-P-2)
 - "RENDER_FAILED" → 503 Service Unavailable (R-READ-01-P-3)
 - URL present → continue

Step 3 – Object store fetch

- InvoicePdfService.stream(pdf_url):
 - Reads from S3-compatible store (or local FS in dev)
 - Streams directly to client (no buffering in READ-01 memory)
 - Supports range requests for partial downloads

Step 4 – Audit log

- Asynchronously write invoice_download_log row
- Does NOT block the response

Step 5 – Return PDF stream

- Content-Type: application/pdf
- Content-Disposition: attachment; filename="{legal_invoice_number}.pdf"
- ETag: derived from invoice_id + pdf_url version
- Cache-Control: private, max-age=31536000, immutable

Cache invalidation flow

Event arrives on Kafka (e.g., sophix.billing.invoice.cancelled, or sophix.notification.pdf.ready from NOT-01)

Step 1 – InvoiceLifecycleEventConsumer reads the event

- Extracts invoice_id, customer_id, operator_code

Step 2 – InvoiceCacheInvalidator.invalidate(invoice_id, customer_id):

- DELETE invoice:detail:{invoice_id}
- DELETE invoice:status:{invoice_id}
- SCAN + DELETE invoice:list:{customer_id}:* (all list pages for this customer)

Step 3 – Ack the Kafka event

Data model

invoice_download_log

One row per PDF download for audit. Append-only.

Column	Type	Notes
id	TEXT PK	ULID
invoice_id	TEXT NOT NULL	FK to invoice
requesting_user_id	TEXT NOT NULL	Customer ID or admin user ID from JWT
requesting_scope	TEXT NOT NULL	CUSTOMER / ADMIN / INTEGRATION
integration_id	TEXT NULL	If scope = INTEGRATION
downloaded_at	TIMESTAMP TZ NOT NULL	
source_ip	TEXT NULL	Client IP (may be NULL behind certain proxies)

Column	Type	Notes
user_agent	TEXT NULL	Truncated to 500 chars
bytes_served	BIGINT NULL	For bandwidth reporting; NULL if range request partial
range_request	BOOLEAN NOT NULL DEFAULT FALSE	Whether this was a partial range request

Indexes: (invoice_id, downloaded_at DESC), (requesting_user_id, downloaded_at DESC), (downloaded_at) for reporting. P10Y retention.

Sample data:

id	invoice_id	requesting_user_id	requesting_scope	downloaded_at	source_ip	bytes_served
dl_01HZ...A	inv_01HZ...	cust_01HX...	CUSTOMER	2026-05-17 14:23:12	(customer IP)	87,234
dl_01HZ...B	inv_01HZ...	admin_user_42	ADMIN	2026-05-17 14:24:01	(internal IP)	87,234
dl_01HZ...C	inv_01HZ...	integration_acme_corp	INTEGRATION	2026-05-17 14:25:00	(integration IP)	87,234

How to read: the same invoice was downloaded once by the customer themselves, once by a support agent helping the customer over the phone, and once by a B2B integration aggregating the customer's invoices. All three are auditable events; finance can reconstruct who accessed what.

API surface — what READ-01 exposes

Mechanism	Used for	Examples
REST (customer-facing — read)	Self-service portal and mobile app reads	GET /api/invoices, GET /api/invoices/{id}, GET /api/invoices/{id}/pdf
REST (admin-facing — read)	Admin console reads with broader scope	Same endpoints, scoped to operator(s) via admin JWT
REST (integration — read)	B2B / accounting integration reads	Same endpoints, scoped to allowed customer_ids via integration JWT
REST (admin operations)	Cache invalidation, re-render triggers, reporting	POST /api/admin/cache/invalidate, POST /api/admin/invoices/{id}/rerender-pdf
Kafka consumption (events in)	Cache invalidation on lifecycle and PDF-ready events	InvoiceCancelled, PaymentApplied, CreditNoteIssued, TaxInvoiceSigned, PdfReady, etc.
Kafka emission (events out)	Admin-triggered re-render	PdfRerenderRequested (consumed by DD_NOT-01 Notification Service)
Object storage reads	PDF streaming from object store	S3-compatible API

What READ-01 does NOT expose:

Capability	Where it lives instead
Generation (any invoice type)	BIL-02-GEN-01 (cycle, one-off, credit/debit notes, pro forma), BIL-02-TAX-01 (tax invoices)
Adjustment proposal/approval	BIL-02-ADJ-01 Invoice Adjustments
Lifecycle transitions (cancel, mark paid)	BIL-02-STA-01 Status Management & Recovery
PDF rendering (template, layout, localization)	DD_NOT-01 Notification Service
Email / SMS / WhatsApp / Telegram / push delivery	DD_NOT-01 Notification Service
Per-channel templates	DD_NOT-01 Notification Service
Payment recording or balance updates	BIL-01 Charging Engine
Wallet operations	BIL-05 Wallet and Topup

REST endpoints

```
# Unified read endpoints (filtered by JWT scope — same path for customer / admin / integration)
GET /api/invoices           ← list with filters + pagination
GET /api/invoices/{id}      ← detail (full structure)
GET /api/invoices/{id}?lightweight=true  ← detail header only
GET /api/invoices/{id}/pdf  ← PDF stream
GET /api/invoices/{id}/status  ← lightweight status (status + outstanding_amount)

# Type-specific convenience (filtered by invoice_type_code)
```

```

GET    /api/invoices/cycle           ← cycle invoices only
GET    /api/invoices/tax         ← tax invoices only
GET    /api/invoices/credit-notes ← credit notes only
GET    /api/invoices/debit-notes ← debit notes only
GET    /api/invoices/pro-forma   ← pro forma documents only

# Admin operations
GET    /api/admin/invoices/download-stats?operator={code}
GET    /api/admin/invoices/pdf-render-failures?operator={code}
POST   /api/admin/invoices/{id}/rerender-pdf           ← emits PdfRerenderRequested to NOT-01
POST   /api/admin/cache/invalidate

```

Request / response samples

List — customer-scoped

Request (customer in self-service portal asks for their unpaid invoices, sorted newest first, page 1):

```

GET /api/invoices?status=PENDING&sort=issue_date_desc&limit=20 HTTP/1.1
Host: api.sophix.example
Authorization: Bearer <customer-JWT>
Accept: application/json

```

The customer JWT carries `customer_id` as a claim. READ-01 derives the filter from the JWT — the customer cannot ask for other customers' invoices.

Response (200 OK):

```

{
  "items": [
    {
      "id": "inv_01HZQX8N5V8KJEC9M3T2A4DPYW",
      "document_kind": "INVOICE",
      "invoice_type_code": "CYCLE_POSTPAID",
      "legal_invoice_number": "WIK-INV-2026-00001847",
      "issue_date": "2026-05-01T00:15:32Z",
      "due_date": "2026-05-15T23:59:59Z",
      "period_start": "2026-04-01T00:00:00Z",
      "period_end": "2026-04-30T23:59:59Z",
      "total_amount": 6402.50,
      "outstanding_amount": 6402.50,
      "currency": "KES",
      "status": "PENDING",
      "payment_status": "UNPAID",
      "pdf_available": true,
      "subscription_id": "sub_01HX..."
    },
    {
      "id": "inv_01HZQX8N5V8KJEC9M3T2A4DQRZ",
      "document_kind": "INVOICE",
      "invoice_type_code": "ONE_OFF",
      "legal_invoice_number": "WIK-INV-2026-00001849",
      "issue_date": "2026-05-03T11:22:01Z",
      "due_date": "2026-05-17T23:59:59Z",
      "period_start": null,
      "period_end": null,
      "total_amount": 1200.00,
      "outstanding_amount": 1200.00,
      "currency": "KES",
      "status": "PENDING",
      "payment_status": "UNPAID",
      "pdf_available": true,
      "subscription_id": "sub_01HX..."
    }
  ],
  "next_cursor": null,
  "total_count": 2
}

```

`next_cursor: null` means no more pages. `total_count` is capped at 10,000; beyond that it's null (no expensive COUNT query).

List — admin-scoped with broader filters

Request (admin filtering across an operator for a specific customer in a date range):

```

GET /api/invoices?operator_code=WIK&customer_id=cust_01HX...&issue_date_from=2026-01-01&issue_date_to=2026-05-31&limit=50
Host: api.sophix.example
Authorization: Bearer <admin-JWT>
Accept: application/json

```

The admin JWT carries operator_codes and roles. READ-01 validates the requested operator_code is within the admin's allowed operators.

Response same shape as above; CANCELLED invoices included by default for admin (R-READ-01-L-5).

Detail — invoice

Request:

```
GET /api/invoices/inv_01HZQX8N5V8KJEC9M3T2A4DPYW HTTP/1.1
Host: api.sophix.example
Authorization: Bearer <customer-JWT>
Accept: application/json
```

Response (200 OK, customer scope — admin-only fields redacted):

```
{
  "id": "inv_01HZQX8N5V8KJEC9M3T2A4DPYW",
  "document_kind": "INVOICE",
  "invoice_type_code": "CYCLE_POSTPAID",
  "legal_invoice_number": "WIK-INV-2026-00001847",
  "operator_code": "WIK",
  "customer_id": "cust_01HX...",
  "subscription_id": "sub_01HX...",
  "billing_mode": "POSTPAID",
  "issue_date": "2026-05-01T00:15:32Z",
  "due_date": "2026-05-15T23:59:59Z",
  "period_start": "2026-04-01T00:00:00Z",
  "period_end": "2026-04-30T23:59:59Z",
  "currency": "KES",
  "subtotal_amount": 5520.26,
  "tax_amount_total": 882.24,
  "total_amount": 6402.50,
  "outstanding_amount": 6402.50,
  "status": "PENDING",
  "payment_status": "UNPAID",
  "grouping": {
    "dimension": "WALLET",
    "key_values": ["INTERNET", "TV"]
  },
  "customer_snapshot": {
    "display_name": "Jane Doe",
    "billing_address_line_1": "...",
    "billing_address_city": "Nairobi",
    "billing_address_country": "KE",
    "tax_identifier": {
      "type": "KRA_PIN",
      "value": "A001234567B"
    }
  },
  "line_items": [
    {
      "id": "il_01HZ...",
      "kind": "SUMMARY",
      "service_category_code": "INTERNET",
      "package_ref": "FIBER_50M",
      "wallet_type_code": "INTERNET",
      "description": "Internet – Fiber 50Mbps (Apr 2026)",
      "subtotal": 3000.00,
      "tax_total": 502.20,
      "total": 3502.20,
      "tax_breakdown": [
        { "tax_code": "EXCISE", "rate_percent": 15.0, "base": 3000.00, "amount": 450.00 },
        { "tax_code": "VAT", "rate_percent": 16.0, "base": 3450.00, "amount": 552.00 }
      ]
    },
    {
      "id": "il_01HZ...",
      "kind": "SUMMARY",
      "service_category_code": "TV",
      "package_ref": "TV_BASIC",
      "wallet_type_code": "TV",
      "description": "TV – Basic Package (Apr 2026)",
      "subtotal": 2520.26,
      "tax_total": 380.04,
      "total": 2900.30,
      "tax_breakdown": [
        { "tax_code": "VAT", "rate_percent": 16.0, "base": 2520.26, "amount": 403.24 }
      ]
    }
  ],
  "tax_summary": [
```

```

    { "tax_code": "EXCISE", "amount": 450.00 },
    { "tax_code": "VAT", "amount": 955.24 }
  ],
  "pdf_status": "AVAILABLE",
  "pdf_url_endpoint": "/api/invoices/inv_01HZQX8N5V8KJEC9M3T2A4DPYW/pdf",
  "related_documents": [
    {
      "id": "inv_01HZQX8N5V8KJEC9M3T2A4DTAX",
      "invoice_type_code": "TAX_INVOICE",
      "legal_invoice_number": "WIK-T-2026-00000451",
      "total_amount": 6402.50,
      "relationship": "TAX_INVOICE_FROM_PAYMENT"
    }
  ],
  "status_history": [
    { "status": "GENERATED", "transitioned_at": "2026-05-01T00:15:32Z", "transitioned_by": "system:cycle_close" },
    { "status": "PENDING", "transitioned_at": "2026-05-01T00:15:33Z", "transitioned_by": "system" }
  ]
}

```

Notes on the response:

- `pdf_url_endpoint` is the READ-01 endpoint to stream the PDF — NOT the direct object-storage URL (clients must authenticate through READ-01 to download)
- `related_documents` surfaces the document graph — the tax invoice that was generated when this cycle invoice was paid is linked back here
- `customer_snapshot` is filtered per the consumer's role; admin-scoped requests see more fields
- Admin-only fields (`triggering_event_ref`, internal metadata, audit metadata) are absent from this customer-scope response

Detail — pro forma

Request: same path, different document kind comes back:

```

GET /api/invoices/pf_01HZX9M2K3LA8N7QR4S5T6U7VW HTTP/1.1
Authorization: Bearer <customer-JWT>

```

Response (200 OK, abridged):

```

{
  "id": "pf_01HZX9M2K3LA8N7QR4S5T6U7VW",
  "document_kind": "PRO_FORMA",
  "invoice_type_code": "PRO_FORMA",
  "pro_forma_number": "WIK-PF-2026-04-00012834",
  "operator_code": "WIK",
  "customer_id": "cust_01HX...",
  "subscription_id": "sub_01HX...",
  "billing_mode": "PREPAID",
  "issue_date": "2026-04-28T03:00:00Z",
  "cycle_start": "2026-05-01T00:00:00Z",
  "cycle_end": "2026-05-31T23:59:59Z",
  "currency": "KES",
  "subtotal_amount": 3879.31,
  "tax_amount_total": 620.69,
  "total_amount": 4500.00,
  "status": "PRESENTED",
  "presented_at": "2026-04-28T03:00:00Z",
  "superseded_at": null,
  "superseded_by_id": null,
  "fulfilled_at": null,
  "cancelled_at": null,
  "line_items": [ ... ],
  "tax_summary": [ ... ],
  "customer_snapshot": { ... },
  "pdf_status": "UNAVAILABLE",
  "pdf_unavailable_reason": "OPERATOR_PRO_FORMA_PDF_DISABLED"
}

```

Pro forma differs from invoice: `pro_forma_number` (non-gap-free) instead of `legal_invoice_number`; `cycle_start/cycle_end` instead of `period_start/period_end`; lifecycle fields `presented_at` / `superseded_at` / `superseded_by_id` / `fulfilled_at` / `cancelled_at` instead of `due_date/payment_status`. A pro forma in SUPERSEDED status would have `superseded_at` populated and `superseded_by_id` pointing to the newer pro forma that replaced it.

Status — lightweight

Request:

```
GET /api/invoices/inv_01HZQX8N5V8KJEC9M3T2A4DPYW/status HTTP/1.1
Authorization: Bearer <customer-JWT>
```

Response (200 OK):

```
{
  "id": "inv_01HZQX8N5V8KJEC9M3T2A4DPYW",
  "status": "PENDING",
  "payment_status": "PARTIALLY_PAID",
  "outstanding_amount": 2902.50,
  "currency": "KES",
  "due_date": "2026-05-15T23:59:59Z"
}
```

Used by mobile apps polling for balance changes — minimal payload, 30-second cache TTL.

PDF — success

Request:

```
GET /api/invoices/inv_01HZQX8N5V8KJEC9M3T2A4DPYW/pdf HTTP/1.1
Authorization: Bearer <customer-JWT>
Accept: application/pdf
Range: bytes=0-65535
```

Response (206 Partial Content — range request honored):

```
HTTP/1.1 206 Partial Content
Content-Type: application/pdf
Content-Disposition: attachment; filename="WIK-INV-2026-00001847.pdf"
Content-Range: bytes 0-65535/87234
Content-Length: 65536
ETag: "inv_01HZ...v1"
Cache-Control: private, max-age=31536000, immutable

<binary PDF bytes>
```

The mobile client fetches the rest of the bytes via subsequent range requests.

PDF — pending (race window)

Request: same as above, but called immediately after invoice issuance (NOT-01 hasn't finished rendering yet).

Response (202 Accepted):

```
HTTP/1.1 202 Accepted
Content-Type: application/json

{
  "pdf_status": "PENDING",
  "retry_after_seconds": 5,
  "message": "PDF rendering in progress. Try again shortly."
}
```

Clients are expected to wait or poll. Retry-After: 5 is also set as a response header.

PDF — unavailable by design

Request: pro forma PDF for an operator that doesn't render pro forma PDFs.

Response (200 OK — not an error; the document exists, just not as a PDF):

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "pdf_status": "UNAVAILABLE",
  "reason_code": "OPERATOR_PRO_FORMA_PDF_DISABLED",
  "message": "This operator does not produce PDF artifacts for pro forma documents. Pro forma details are available via the detail endpoint."
}
```

PDF — render failed irrecoverably

Response (503 Service Unavailable):

```
HTTP/1.1 503 Service Unavailable
Content-Type: application/json

{
  "pdf_status": "RENDER_FAILED",
  "reason_code": "TEMPLATE_RENDER_ERROR",
  "message": "PDF rendering failed; operations team has been notified. The invoice data is fully available via the detail endpoint."
}
```


Admin can trigger re-render via POST /api/admin/invoices/{id}/rerender-pdf (R-READ-01-O-2) which emits PdfRerenderRequested to NOT-01.

Errors

Standard envelope across all endpoints:

```
{
  "error_code": "INVOICE_NOT_FOUND",
  "message": "Invoice not found or not accessible.",
  "request_id": "req_01HZ..."
}
```

Common codes:

- 401 Unauthorized — missing/invalid JWT (error_code: AUTH_REQUIRED)
- 403 Forbidden — JWT valid but role insufficient (e.g., integration JWT with no allowed customer_ids matching the request)
- 404 Not Found — invoice doesn't exist OR exists but consumer doesn't have access (intentional 404 not 403 to avoid leaking existence; R-READ-01-A-2)
- 429 Too Many Requests — rate limit exceeded (Retry-After header populated)
- 503 Service Unavailable — PDF render irrecoverably failed

Admin operations — rerender PDF

Request:

```
POST /api/admin/invoices/inv_01HZQX8N5V8KJEC9M3T2A4DPYW/rerender-pdf HTTP/1.1
Authorization: Bearer <admin-JWT>
Content-Type: application/json

{
  "reason": "Customer reported corrupted PDF download; investigating template issue."
}
```

Response (202 Accepted):

```
{
  "request_id": "rerender_01HZ...",
  "invoice_id": "inv_01HZQX8N5V8KJEC9M3T2A4DPYW",
  "rerender_requested_at": "2026-05-18T10:23:45Z",
  "message": "Rerender requested. PdfRerenderRequested event emitted to DD_NOT-01 Notification Service. PDF status will up"
}
```

Cross-DD coordination

Touched	What READ-01 owns	What the other DD owns
BIL-02 Invoicing (Core)	Reads invoice, invoice_line_item, pro_forma, pro_forma_line_item, invoice_history. Reuses Core's customer_snapshot structure.	Owns tables, lifecycle, generator framework.
BIL-02-GEN-01 Invoice Generation	Consumes InvoiceIssued, CreditNoteIssued, DebitNoteIssued events for cache invalidation.	Owns generation.
BIL-02-TAX-01 Tax Invoice & Gateway	Consumes TaxInvoiceIssued, TaxInvoiceSigned, TaxInvoiceCancelled events for cache invalidation. Reads tax invoice rows like any other invoice.	Owns tax invoice lifecycle.
BIL-02-ADJ-01 Invoice Adjustments	Consumes credit/debit note events for invalidation. Serves the resulting notes through the same unified read API.	Owns adjustment workflow.
BIL-02-STA-01 Status Management & Recovery	Consumes InvoiceStatusChanged and InvoiceCancelled events for invalidation. Surfaces status_history in detail responses.	Owns lifecycle transitions.
BIL-01 Charging Engine	Consumes PaymentApplied event for invalidation (payment changes outstanding_amount).	Owns balance + payment.
DD_NOT-01 Notification Service	Reads invoice.metadata.pdf_url (written by NOT-01). Consumes PdfReady / PdfRenderFailed events for cache and status display. Emits PdfRerenderRequested consumed by NOT-01.	Owns PDF rendering, channel templates, and push delivery (email/SMS/WhatsApp/Telegram/push).
Customer self-service portal / mobile app	Provides the consumer-facing REST API.	UI teams own the frontends.

Touched	What READ-01 owns	What the other DD owns
Admin console	Provides the admin REST API.	Admin UI team.
B2B integrations	Provides the integration REST API.	Integration partners own their clients.
Object storage (S3)	Reads stored PDF artifacts. Does NOT write.	NOT-01 writes; infrastructure owns the bucket.
Redis cache	Reads, writes, and invalidates cache keys per FOUNDATION_CACHE.md.	Infrastructure owns Redis.

Dependencies

Dependency	Owner
BIL-02 Invoicing (Core)	billing team (same wave)
BIL-02-GEN-01 Invoice Generation	billing team (same wave)
BIL-02-TAX-01 Tax Invoice & Gateway	billing team (same wave)
BIL-02-ADJ-01 Invoice Adjustments	billing team (same wave)
BIL-02-STA-01 Status Management & Recovery	billing team (same wave)
BIL-01 Charging Engine	billing team
DD_NOT-01 Notification Service	notification team (PDF rendering, object store writes, all push delivery channels)
Customer self-service portal / mobile app	UI team
Admin console	admin UI team
FOUNDATION_DB.md, FOUNDATION_KAFKA.md, FOUNDATION_CACHE.md, FOUNDATION_AUTH.md	infrastructure team
Redis deployment	infrastructure team
S3-compatible object storage	infrastructure team

Operator-configurable capabilities (specific to READ-01)

Capability	Where configured	Notes
Cache TTLs	Per-operator override of defaults	Detail (default 5min), list (default 1min), status (default 30s). Operators with very volatile invoice data may reduce TTLs; operators with stable data may extend them.
Customer field redaction list	Per-operator config	Operator chooses which customer_snapshot fields to expose vs redact in customer-scoped responses
Admin role → field projection mapping	Per-operator config	Maps admin roles (BILLING_ADMIN, CUSTOMER_SUPPORT_AGENT, etc.) to which fields they see
Rate limits	Per-operator override	Default 100 req/min customer; operators with B2B-heavy traffic may raise integration limits
Include-cancelled default	Per-operator config	Default false (hide cancelled invoices); operator can flip if their UX wants cancelled visible
Download log retention	Per-operator override	Default 10 years; operators in lighter-regulation markets may reduce

B — Current TZ

A unified invoice read API does not exist as a first-class concept in TZ Sophix. Various per-channel endpoints exist scattered across the codebase, but there is no consolidated read service with caching, ACL, or audit to describe or compare against.

C — Gap & bridge

READ-01 is entirely new functionality. The capability described in section A — unified API across consumers, event-driven Redis caching, NOT-01-rendered PDF serving, centralized access control, download audit logging — is introduced fresh by V3.